Camera Transport in the MABEL Real-Robot Control Center: from MJPEG `<img>` to H.264/WebCodecs

MABEL project

2026-07-13

Abstract

The MABEL Control Center is the web studio served from the robot host (`thor`) through which a remote operator drives the physical robot. Its camera views were the last client in the fleet still rendered as native MJPEG `<img>` elements, while the iOS app and the website teleop page had already moved to the camera server's hardware-encoded H.264 stream. The studio was correspondingly slow, most severely over the public relay. We move every camera surface in the studio onto H.264 decoded in the browser with WebCodecs, keeping MJPEG strictly as a fallback, and we remove three latency accumulators specific to this studio: a streaming-hostile HTTP middleware, an unbounded decoder queue, and an availability probe that leaked MJPEG streams. Measured in Chrome on macOS at 1280×720@30 fps: the browser-side pipeline runs at 1.3 ms median (arrival $\rightarrow$ painted) at the full source frame rate, and the wire cost of the four-tile camera strip falls from 36.8 Mbps to 13.0 Mbps (2.8×).

Keywords: teleoperation, video transport, H.264, WebCodecs, MJPEG, latency, bounded queues, robot web interfaces.

1 Overview

The robot's camera server (`hardware_bridge/mabel_hw/cameras/video_server.py`) already exposes each camera on two planes: a multipart MJPEG stream at `/camera/<name>/stream.mjpg`, and the same frames hardware-encoded once (NVENC on the Jetson) and published as H.264 Annex-B access units over a WebSocket at `/camera/<name>/h264` — one binary message per access unit. The iOS app and the website teleop page consume the H.264 plane. The Control Center did not: every camera surface (the four-tile strip, add-window tiles, the first-person view, and the textures projected into the 3-D scene) pointed an `<img>` at the MJPEG plane.

MJPEG is intra-only: every frame is a complete JPEG. H.264 is inter-frame, so at equal resolution it costs roughly a quarter as much on the wire (§3), and the difference is decisive on the relay path, where the studio is reached over the public internet through `rathole`+Caddy.

2 Method

2.1 Transport selection

Let r denote whether the page is served through the relay and k whether it carries an app token (`?key=`). The H.264 socket URL is

$$u(\texttt{name}) = \begin{cases} \texttt{ws://<host>:8080/camera/name/h264} & \neg r \quad \text{(LAN: straight to the camera server)} \\ \texttt{wss://<relay>/camera/name/h264?key=} & r \wedge k \quad \text{(the gated main-domain route)} \\ \texttt{wss://<page>/ws/cam/name/h264} & r \wedge \neg k \quad \text{(our same-origin proxy)} \end{cases}$$

The third case is the normal deployment (`control.mabel.example.com` with no key): the gated route would reject us, so the studio backend relays the same WebSocket, riding the page's own origin and session cookie. The proxy is binary-transparent — one upstream message is forwarded as exactly one downstream message — so the browser decoder is identical on all three paths.

2.2 Decoding

Each access unit is split into NAL units on Annex-B start codes. Parameter sets (SPS, type 7; PPS, type 8) are cached and, when they change, an `AVCDecoderConfigurationRecord` (`avcC`) is assembled and handed to `VideoDecoder.configure` with `optimizeForLatency`. VCL NALs (types 1 and 5) are re-framed as length-prefixed AVCC and submitted as one `EncodedVideoChunk` per frame; decoded frames are drawn to a `<canvas>` inside the decoder's output callback, so display costs one decode and no jitter buffer.

The encoder contract this depends on is: baseline profile, `yuv420p`, Annex-B, one access-unit delimiter per access unit, and SPS+PPS in-band at every IDR (so a late joiner configures at the next keyframe, $\approx 1\,\text{s}$).

2.3 Bounding latency: everything that can queue, drops

Latency in this pipeline does not appear as slowness; it appears as a queue. Three of them had no bound:

1. **The decoder.** WebCodecs never sheds load: it decodes whatever it is given. If the decoder is slower than the stream, `decodeQueueSize` grows monotonically and the picture falls further behind real time every frame — video that is perfectly smooth and seconds late. We bound it: past a depth of $Q_d = 2$ frames we stop feeding and resync at the next IDR. Because dropping a delta frame breaks the GOP, the drop must skip *to a keyframe* rather than feed a reference-less frame.

2. **The relay proxy.** If the proxy awaited each `send` to the browser from inside its upstream read loop, a viewer on a slow link would stall that read, and the camera server's own 120-deep per-client queue ($\approx 4\,\text{s}$ at $30\,\text{fps}$) would fill behind it. The proxy instead reads upstream at full speed into a shallow queue ($Q_p = 4$, `CAM_WS_QUEUE`) and, on overflow, discards the backlog and resyncs at the next IDR. Clients that reach the camera server directly (iOS, the website) never traverse this hop, which is why the delay was studio-specific.

3. **Departed viewers.** The backend stamped cache headers with a Starlette `BaseHTTPMiddleware` (`@app.middleware("http")`), which swallows the ASGI

`http.disconnect` message for *streaming* responses. The MJPEG pass-through therefore never learned that its viewer had left and kept reading the camera server forever — one leaked upstream reader per departed tab. The middleware is now pure ASGI (it wraps `send` only) and the pump checks `request.is_disconnected()` per chunk.

A fourth drain was not a queue but a leak: the strip probed for a depth stream by pointing an `<img>` at `<name>_depth/stream.mjpg`. That opens a full multipart stream which cannot be reliably aborted (WebKit ignores `removeAttribute("src")`), so up to four hidden depth streams could run for the life of the page, competing with the video the operator is actually watching. Availability is now probed with the one-shot `/api/cam/<name>/frame.jpg`, which returns a single JPEG (or 404) and closes.

2.4 Fallback policy

MJPEG remains as the fallback (no WebCodecs, no `/h264` route on an older camera build, or a stalled stream), but the two transports are never run concurrently — that would pay the very bandwidth we are escaping. The MJPEG `<img>` is attached only if H.264 fails to deliver: on a cold start the fallback timer is deferred while *bytes* are still arriving on the H.264 socket (the decoder may legitimately still be settling), and it fires promptly when the socket is silent. Not opening the stream in the common case also sidesteps the WebKit abort problem above.

3 Experiments

`tools/exp_video_latency.py` regenerates every number and the figure below. It serves a real H.264 stream (libx264, baseline, single slice per frame, AUD-delimited, SPS/PPS in-band at each IDR — the production contract) paced at real time from a fake camera server, drives the studio's *actual* modules (`web/js/h264.js`, `web/js/camview.js`) in Chrome, and reports latency from the decoder's own chunk timestamps. Latency is defined per frame as

$$\ell = t_{\text{painted}} - t_{\text{bytes arrived in the browser}},$$

i.e. the portion of glass-to-glass that the studio owns; capture, encode and network are shared with the iOS and web clients and are measured separately in `server/tools/exp_h264_video.py`. Source: 1280×720@30 fps, six seconds per case, Chrome on macOS (Apple silicon).

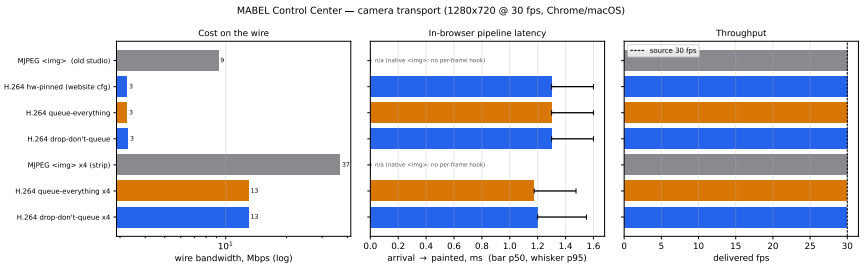


Figure 1: Camera transport in the studio. Left: cost on the wire (log scale) — MJPEG's intra-only coding is what saturates the relay. Middle: in-browser pipeline latency; a native MJPEG `<img>` exposes no per-frame hook, so no latency is reported for it rather than a fabricated one. Right: delivered frame rate against the 30 fps source.

Case	fps	Mbps	ℓ p50 (ms)	ℓ p95 (ms)	backend
MJPEG `<img>` (old studio)	30.0	9.2	n/a	n/a	—
H.264, queue everything	30.0	3.3	1.3	1.6	hardware
H.264, drop-don't-queue	30.0	3.3	1.3	1.6	hardware
MJPEG `<img>` ×4 (strip)	30.0	36.8	n/a	n/a	—
H.264, queue everything ×4	30.0	13.0	1.2	1.5	hardware
H.264, drop-don't-queue ×4	30.0	13.0	1.2	1.5	hardware

Table 1: 1280×720@30 fps, 6 s per case. The four-stream rows are the studio's permanent camera strip. The drop guard costs nothing when the pipeline is healthy (zero drops, peak decoder queue zero); it exists to bound the unhealthy case.

Findings.

- **Decode is not the bottleneck.** The browser pipeline runs at 1.3 ms median and holds the full 30 fps even with four simultaneous 720p streams. Measured in isolation, hardware decode sustains 3607 fps and software 1102 fps at 720p — both far above real time.
- **The wire is the bottleneck.** The four-tile strip costs 36.8 Mbps as MJPEG and 13.0 Mbps as H.264. On a LAN the former is merely wasteful; over the relay it does not fit, and the excess appears to the operator as lag.
- **The drop guard is free when healthy.** In all nominal cases it recorded zero drops and a peak decoder queue of zero; it changes behaviour only when the pipeline is already falling behind, where it converts unbounded delay into a brief skip.

A measurement error worth recording. Our first fixture encoded the test stream with `-tune zerolatency`, which silently enables x264 *sliced-threads*, splitting each frame into several slices. An access-unit splitter that closes at the first VCL NAL then tears single frames into fragments. Feeding those fragments to `VideoDecoder` produced `EncodingError: Decoding error` on every hardware-decode configuration while software decode tolerated them, which we very nearly recorded as "Chrome on macOS cannot hardware-decode H.264." With a production-faithful stream (one slice per frame, split on access-unit delimiters exactly as the camera server does) hardware decode is flawless (180/180 frames). The lesson is that a decoder benchmark is only as trustworthy as the bitstream it is fed; the harness now pins `sliced-threads=0`, `slices=1` and `aud=1`.

4 Implementation and usage

- `web/js/h264.js` — `H264Stream`: NAL splitting, `avcC` construction, the decode-backend ladder, the drop-don't-queue guard, and `stats()` (fps, drops, queue depth, latency percentiles).
- `web/js/camview.js` — `CamView`: one camera surface; owns the transport choice, the canvas/`<img>` swap, the fallback timer, and reconnection. Every camera in the studio goes through it.
- `server/app.py` — `/ws/cam/<name>/h264` (the relay proxy, with the shallow queue), the disconnect-aware MJPEG pass-through, and the pure-ASGI `_NoCache` middleware. `server/config.py:CAM_WS_QUEUE` sets Q_p.

- Tests: `tests/test_video_h264.py` (byte-identical pass-through, upstream reaped when the viewer leaves, clean close when the camera is offline, and the route is login-gated); `tests/js/h264.test.mjs` (the Annex-B/`avcC` parsing the decoder depends on).
- Reproduce: `python3 tools/exp_video_latency.py`; rebuild this document with `make -C docs`.

5 Limitations

- Latency here is the *in-browser* segment only. Capture, encode, and the relay network are not included; the end-to-end figure is the sum of this and the server-side budget measured in `server/tools/exp_h264_video.py`.
- Measurements are from a synthetic `testsrc2` pattern on one machine (Chrome, Apple silicon). The robot's NVENC bitstream and a real scene will differ in bitrate; the *ratio* between transports is the transferable result, not the absolute Mbps.
- MJPEG latency is reported as n/a rather than estimated: a native `<img>` on `multipart/x-mixed-replace` offers no per-frame hook. Comparing the transports on latency alone would require re-implementing MJPEG as a fetch-painter, which the studio no longer uses on the fast path.
- The camera server's own per-client queue (120 access units) is unchanged and is shared with the iOS and website clients; it remains the largest single buffer in the system and is the natural next target.
- Depth streams have no H.264 encoder, so toggling a tile to depth falls back to MJPEG by design.

References

1. W3C, *WebCodecs*, W3C Working Draft. `https://www.w3.org/TR/webcodecs/`
2. ITU-T Rec. H.264, *Advanced video coding for generic audiovisual services.*
3. ISO/IEC 14496-15, *Carriage of NAL unit structured video* (the `avcC` record).
4. MABEL, `server/docs/server.pdf`, §H.264 — the server-side video budget and the UDP/WebSocket transport split.